

API Mocking in Playwright

What is API Mocking?

API Mocking means **creating a fake API response instead of calling the real server**.

Instead of sending a request to the actual backend, Playwright **intercepts the request** and returns the data that we define.

Think of it like this:

 *Mocking = Pretending the backend already replied with the data you want.*

Real Life Example

Imagine you are testing an online shopping website.

Normally the browser sends a request like this:

```
Browser
  |
  | GET /products
  ▼
Real Server
  |
  ▼
Returns Product List
```

With API Mocking:

```
Browser
  |
  | GET /products
  ▼
Playwright intercepts the request
  |
  ▼
Returns Fake Product List
```

The real server is **never contacted**.

Why do we use API Mocking?

Suppose the backend team hasn't finished developing the API.

Without mocking:

✗ Your UI cannot display any data.

With mocking:

✓ You can continue testing the UI using fake data.

Benefits of API Mocking

- ✓ No dependency on backend developers
 - ✓ Faster test execution
 - ✓ No internet required
 - ✓ Predictable test data
 - ✓ Easier to test edge cases
 - ✓ Can simulate server errors (404, 500, etc.)
-

Example Application

Suppose the application loads products using:

```
GET https://example.com/api/products
```

Normally the API returns

```
[
  {
    "id": 1,
    "name": "Laptop",
    "price": 50000
  },
  {
    "id": 2,
    "name": "Mobile",
    "price": 20000
  }
]
```

Instead of calling the real API, Playwright will return our own fake data.

Playwright Example

```
import { test, expect } from '@playwright/test';

test('Mock Products API', async ({ page }) => {

  // Intercept the API request
  await page.route('**/api/products', async route => {

    // Return fake response
    await route.fulfill({
      status: 200,
      contentType: 'application/json',
      body: JSON.stringify([
        {
          id: 101,
          name: 'Playwright Book',
          price: 999
        },
        {
          id: 102,
          name: 'Automation Course',
          price: 1999
        }
      ])
    });

  });

  // Open application
  await page.goto('https://example.com');

});
```

How it Works

Step 1

Application sends request

```
GET /api/products
```



Step 2

Playwright catches the request

```
page.route('**/api/products')
```



Step 3

Instead of going to the server

✗ Request is blocked



Step 4

Playwright sends fake JSON

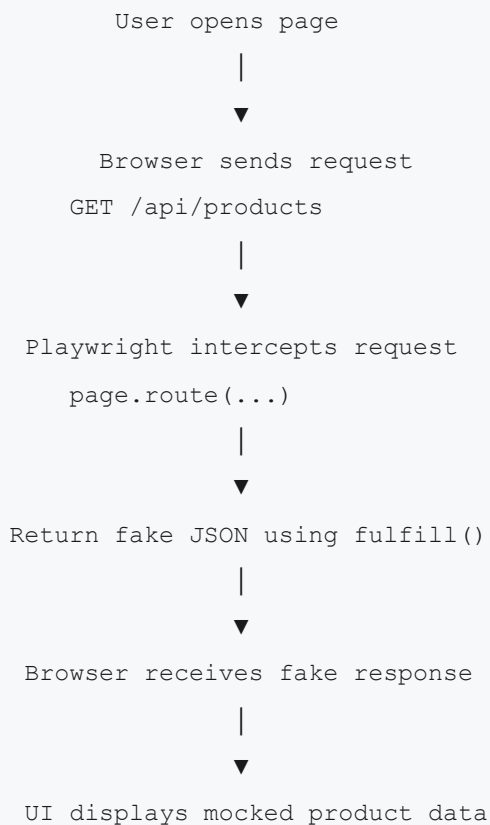
```
[
  {
    "id":101,
    "name":"Playwright Book",
    "price":999
  }
]
```



Step 5

The application believes the response came from the real backend.

Workflow Diagram



Understanding `page.route()`

```
await page.route('**/api/products', async route => {
```

This tells Playwright:

"Whenever any request matches `/api/products`, intercept it."

The `**` is a wildcard that matches any domain.

Examples:

```
https://example.com/api/products
```

```
https://staging.example.com/api/products
```

```
https://test.example.com/api/products
```

All match:

```
**/api/products
```

Understanding `route.fulfill()`

This method creates a fake HTTP response.

```
await route.fulfill({  
  status: 200,  
  contentType: 'application/json',  
  body: JSON.stringify(data)  
});
```

`status`

```
status: 200
```

Pretend the request was successful.

`contentType`

```
contentType: 'application/json'
```

Tell the browser that the response is JSON.

`body`

```
body: JSON.stringify(...)
```

Convert the JavaScript object into JSON and send it back as the response.

Before Mocking

```
graph TD; Browser --> Real_API[Real API]; Real_API --> Database; Database --> Response;
```

Browser

Real API

Database

Response

Problems:

- Backend may be down
 - Slow network
 - Unstable test data
 - Server errors
-

After Mocking

```
graph TD; Browser --> Playwright; Playwright --> Fake_JSON[Fake JSON];
```

Browser

Playwright

Fake JSON

Benefits:

- Fast
 - Stable
 - Reliable
 - Fully controlled data
-

When Should You Use API Mocking?

Use API mocking when:

-  The backend API is not ready.
-  The API is slow or unreliable.
-  You want consistent test data.
-  You need to simulate error responses (404, 500, etc.).
-  You want to test the UI independently of the backend.

Avoid mocking when:

-  You want to verify that the real backend and frontend work together (end-to-end testing).
-  You need to validate actual API responses from the server.

Key Takeaways

- **API Mocking** means replacing a real API response with a fake one.
- `page.route()` intercepts matching network requests.
- `route.fulfill()` returns a custom response without contacting the real server.
- Mocking helps you test the UI even when the backend is unavailable or unstable.
- It makes tests faster, more reliable, and easier to reproduce.

 **Beginner Tip:** Think of Playwright as a "traffic police officer." When the browser tries to contact the server, Playwright stops the request, hands the browser a fake response, and the browser continues as if it came from the real backend.